

DIP PROJECT REPORT

ARTISTIC IMAGE STUDIO

*A MATLAB App Designer Application for
Non-Photorealistic Image Rendering & Artistic Style Transfer*

Digital Image Processing (DIP)
SUBMITTED TO: Mam Amina Amir

SUBMITTED BY: Group#9

Alisha Fatima(2023-ag-10040)

Kinza Fatima(2023-ag-10072)

Laika Butt(2023-ag-10075)

Maryam Zafar(2023-ag-10081)

Aliza Tariq(2023-ag-10041)

MATLAB App Designer Project

Submitted in partial fulfillment of DIP course requirements

Spring 2026

Table of Contents

1. Project Overview	3
2. Objectives	3
3. Tools & Technologies	4
4. Application Architecture	4
5. Image Processing Filters	5
6. Algorithm Deep-Dive	6
7. Before & After Results	7
8. Code Structure	10
9. Conclusion	11

1. Project Overview

Artistic Image Studio is an interactive image processing application developed using MATLAB App Designer as a Digital Image Processing (DIP) course project. The application enables users to upload portrait or general-purpose images and apply a variety of artistic rendering effects inspired by traditional art forms such as pencil sketching, charcoal drawing, ink outlining, oil painting, and pop art.

The app leverages core DIP techniques — including spatial filtering, edge detection, color quantization, and intensity transformations — to transform ordinary photographs into stylized artistic outputs, all within a clean and interactive graphical user interface (GUI).

Project Title	Artistic Image Studio
Platform	MATLAB App Designer (R2021a or later)
Domain	Digital Image Processing (DIP)
Input	JPG / JPEG / PNG image files
Output	Artistic rendered image displayed in-app
Interface	Dual-panel GUI with Upload button and filter Dropdown

2. Objectives

The primary objectives of this project are:

- Implement non-photorealistic rendering (NPR) techniques using MATLAB image processing toolbox functions.
- Design a user-friendly GUI that allows real-time artistic filter application.
- Apply core DIP concepts: spatial filtering, edge detection, intensity mapping, and color quantization.
- Demonstrate the transformation of input photographs into multiple distinct artistic styles.
- Provide a side-by-side preview of original and processed images for comparison.
- Explore the mathematical foundations of image transformations such as dodge blending and bilateral filtering.

3. Tools & Technologies

Filter Name	Algorithm / Technique	Key MATLAB Functions
MATLAB	Primary programming and computational environment	R2021a+
App Designer	GUI development framework built into MATLAB	uifigure, uiaxes
Image Proc. Toolbox	Provides all image processing functions used	imgaussfilt, imadjust
im2double / im2uint8	Precision conversion between image data types	im2double
imshow	Renders images into UIAxes display panels	imshow(img, Parent)

The application is fully self-contained within MATLAB and does not require any external libraries or toolboxes beyond the standard Image Processing Toolbox.

4. Application Architecture

The app is built as a classdef component (DIP < matlab.apps.AppBase) following MATLAB OOP conventions. The architecture is organized into three main layers:

4.1 UI Components

- UIFigure — Main application window (640 x 480 pixels, dark-themed background color [0.149, 0.180, 0.173])
- UIAxes (Preview Panel) — Displays the original uploaded image on the left side
- UIAxes2 (Result Panel) — Displays the artistically processed output on the right side
- UploadImageButton — Opens a native file picker dialog for image selection
- DropDown (Filter Selector) — Lists all 5 available artistic styles; triggers processing on value change

4.2 Callback Methods

- UploadImageButtonPushed — Reads the selected image, stores it in UserData, and displays it in UIAxes
- DropDownValueChanged — Reads the stored image, applies the selected filter algorithm, and displays result in UIAxes2

4.3 Data Flow

Image data flows: File System → uigetfile → imread → UserData → im2double → Filter Algorithm → imshow(UIAxes2). Storing the raw image in UIFigure.UserData avoids re-loading from disk on every filter change.

5. Image Processing Filters

The application implements five distinct artistic style filters. Each filter is selected via the dropdown menu and processed in real-time:

Filter Name	Algorithm / Technique	Key MATLAB Functions
Pencil Sketch	Grayscale conversion → Inversion → Gaussian Blur → Dodge Blend → Normalization → Contrast Stretch	rgb2gray, imgaussfilt, imadjust, mat2gray
Ink Outline	Grayscale → Canny Edge Detection → Morphological Dilation → White Canvas Overlay	edge (Canny), imdilate, strel

Filter Name	Algorithm / Technique	Key MATLAB Functions
Oil Painting	Bilateral Filtering → Color Quantization (8 bins) → Median Filter → Edge Sharpening	imbilatfilt, medfilt2, imsharpen
Charcoal Sketch	Grayscale → Texture via stdfilt → Canny Edges → Shadow Blend → Contrast Adjust	stdfilt, edge (Canny), imdilate, imadjust
Pop Art	Color Quantization (5 levels) → Canny Edge Detection → Black Outline Overlay → Contrast Boost	edge (Canny), imdilate, imadjust

6. Algorithm Deep-Dive

6.1 Pencil Sketch — Dodge Blend Method

This technique replicates the look of a hand-drawn pencil sketch. The core formula is the "Color Dodge" blend mode used in digital art applications:

```
output = gray ./ (1 - blurredInverted + ε) where ε = 0.001
```

- Step 1: Convert to grayscale using `rgb2gray`
- Step 2: Invert pixel intensities (`invImg = 1 - gray`)
- Step 3: Apply strong Gaussian blur ($\sigma = 8$) to simulate pencil shading softness
- Step 4: Apply dodge blend formula to reveal sketch-like edges
- Step 5: Normalize with `mat2gray`, stretch contrast with `imadjust` for clean white paper effect

6.2 Ink Outline — Canny Edge Detection

Produces a black-ink line drawing on a pure white background using the Canny edge detector with a dual-threshold:

- Canny thresholds `[0.08, 0.20]` detect both weak and strong edges for clean line art
- Morphological dilation with `strel('disk',1)` thickens lines to resemble pen strokes
- Edges are projected onto all three RGB channels of a white `(255,255,255)` canvas

6.3 Oil Painting — Bilateral + Quantization

Mimics the appearance of oil paint by smoothing while preserving edges, then reducing color palette:

- `imbilatfilt` (bilateral filter) smooths textures while preserving hard edges ($\sigma = 0.15$, radius = 5)
- Color quantization to 8 discrete bins: `quantized = round(img × 7) / 7`
- Per-channel median filter (5×5) forms cohesive "paint blobs" typical of oil paintings
- `imsharpen` (Radius=1.5, Amount=1.2) adds crisp brush stroke boundaries

6.4 Charcoal Sketch — Texture + Shadow Blend

Creates the rich dark texture of charcoal on paper by combining local texture information with edge strokes:

- `stdfilt` with a disk-shaped neighborhood extracts local standard deviation as a texture map
- Shadow formula: `charcoal = (1-gray)×1.5 + texture×0.5` creates deep tonal variation
- Edge pixels darkened further (`×0.3`) to simulate heavy charcoal strokes on detail areas
- `imadjust` stretches output to `[0.1, 0.8]` for authentic paper-and-charcoal contrast

6.5 Pop Art — Quantization + Outline

Inspired by Andy Warhol's pop art style, this filter creates a bold poster-like look:

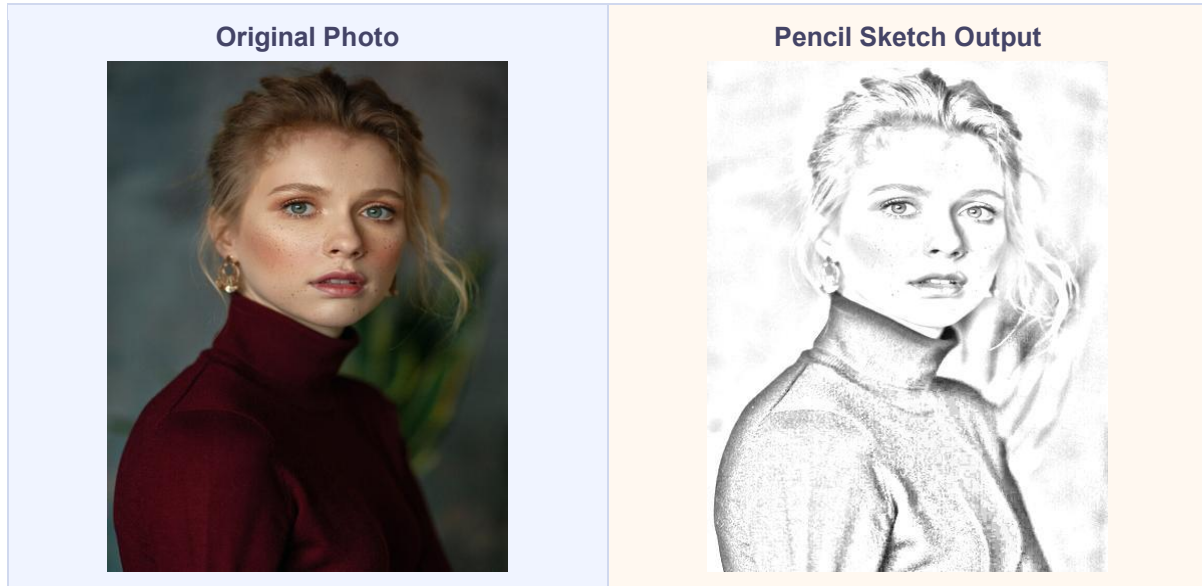
- 5-level color quantization reduces the palette: $\text{floor}(\text{img} \times 5) / 5$ creates flat color regions
- Canny edges (threshold=0.1) with dilation produce thick black contour outlines
- `imadjust` with `stretchlim` maximizes contrast for vivid, high-saturation output

7. Before & After Results

The following pages demonstrate the five artistic filters applied to sample portrait images. Each comparison shows the original photograph alongside the processed artistic output.

7.1 Pencil Sketch Filter

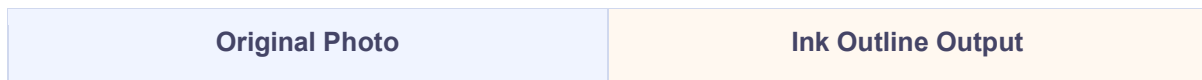
PORTRAIT SAMPLE — PENCIL SKETCH EFFECT

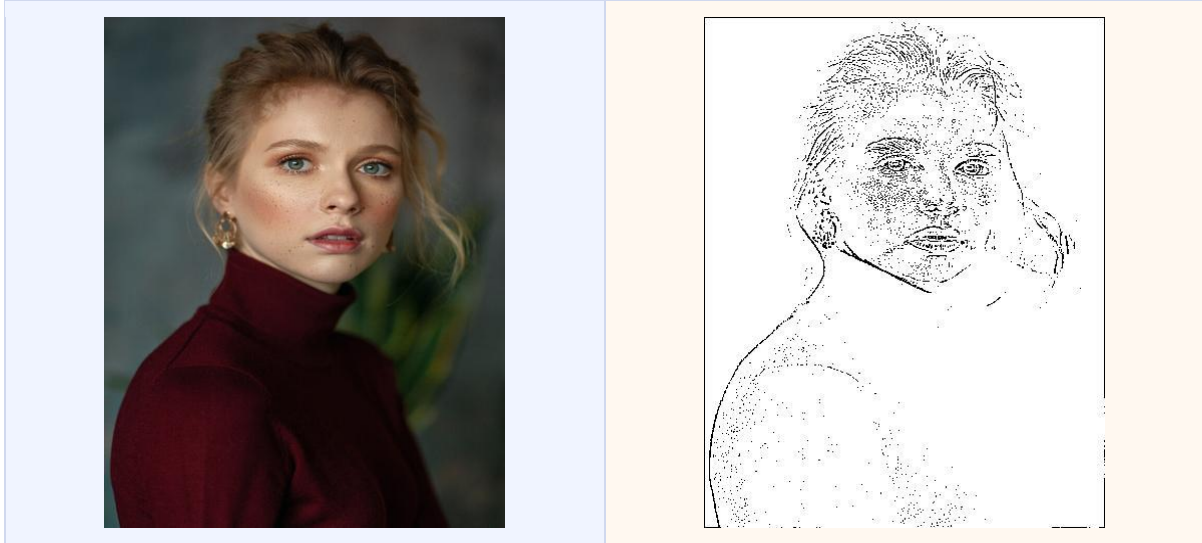


The pencil sketch filter successfully extracts fine facial features — eyebrows, freckles, and hair strands — and renders them as delicate graphite lines on a white paper background. The dodge blend amplifies light-dark contrast, mimicking real pencil shading technique.

7.2 Ink Outline Filter

PORTRAIT SAMPLE — INK OUTLINE EFFECT

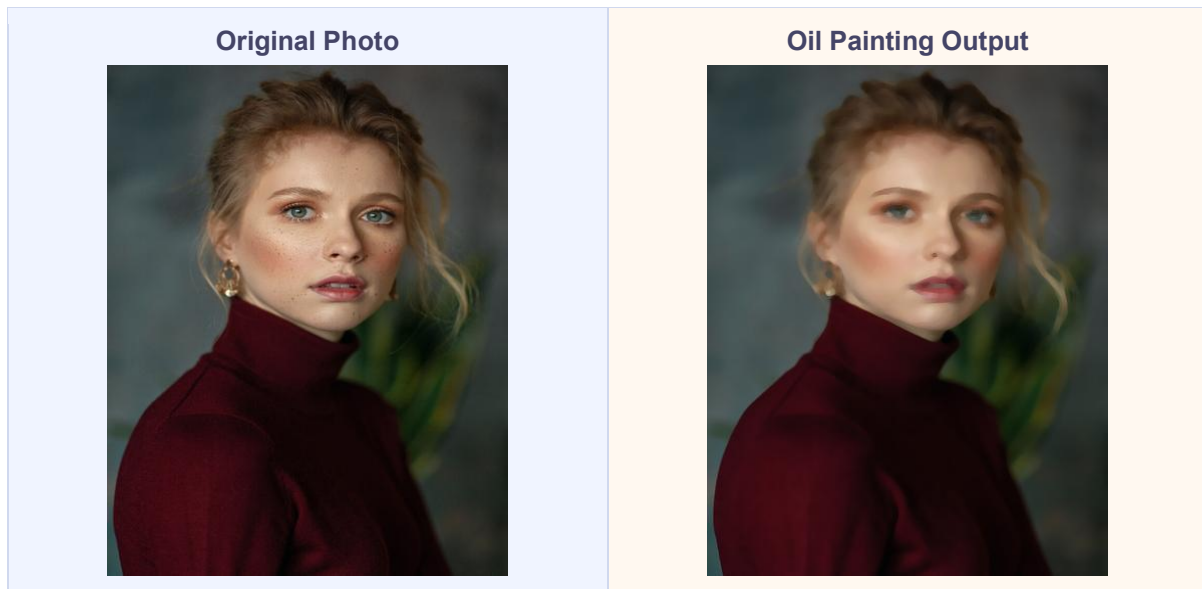




The Canny detector with dual thresholds captures both prominent contours and subtle details such as hair, earrings, and collar edges. The dilation step gives ink-like weight to the strokes while the white background creates a clean line art result.

7.3 Oil Painting Filter

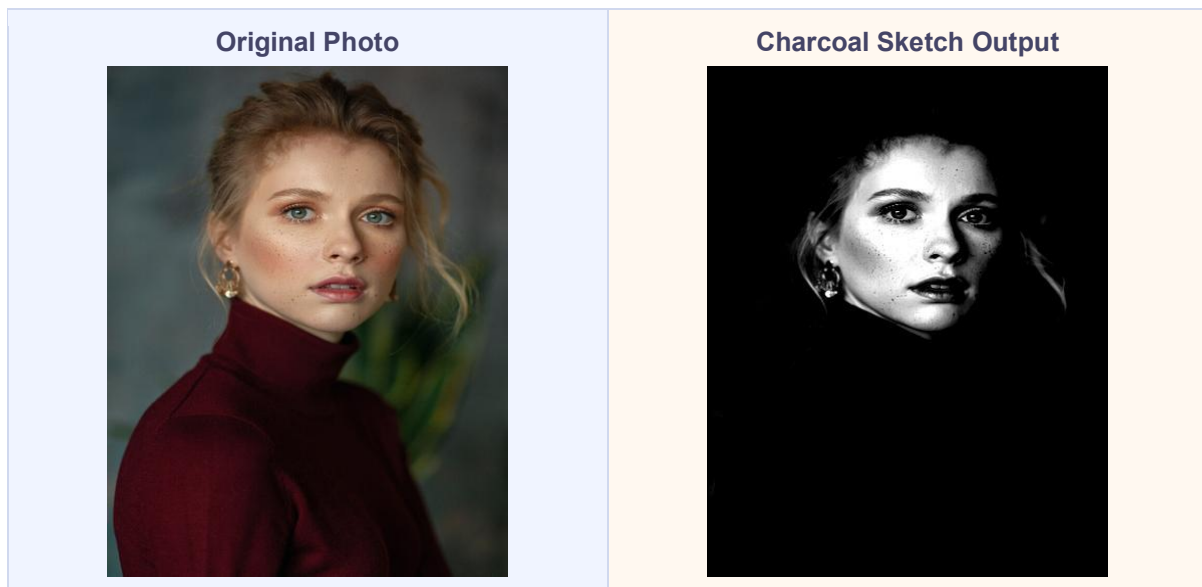
PORTRAIT SAMPLE — OIL PAINTING EFFECT



The bilateral filter effectively smooths skin texture while preserving sharp jawline and eye edges. Color quantization to 8 bins groups similar hues into distinct paint regions. The final imsharpen step adds brushstroke clarity characteristic of oil paintings.

7.4 Charcoal Sketch Filter

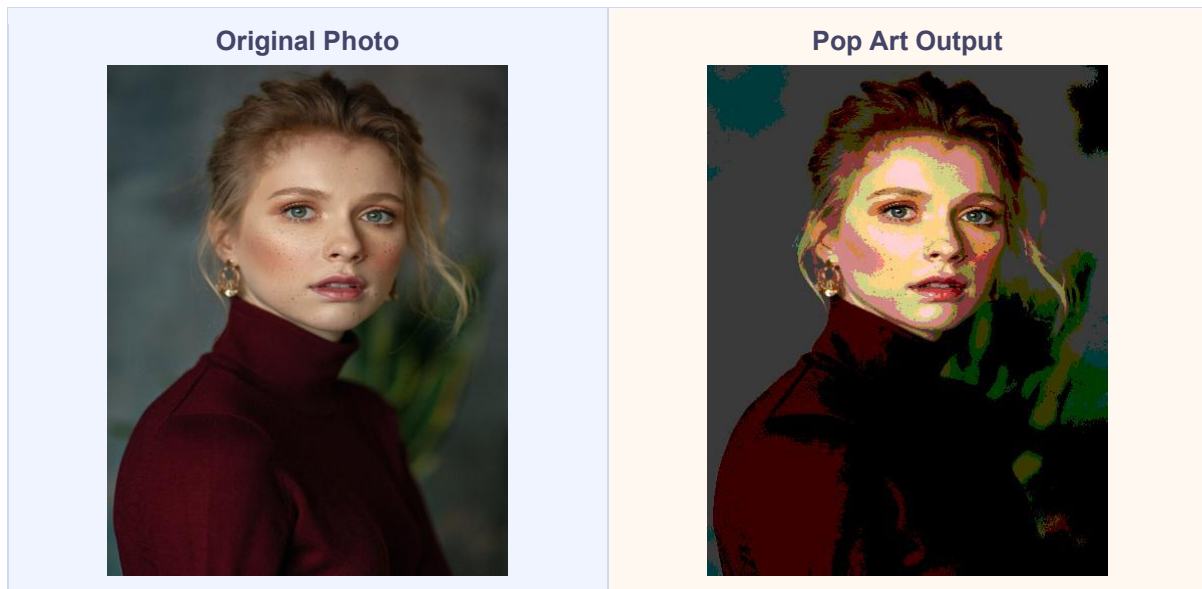
PORTRAIT SAMPLE — CHARCOAL SKETCH EFFECT



The charcoal effect produces dramatic tonal contrast with deep shadows and highlighted areas. The texture map derived from stdfilt adds granularity that mimics the rough texture of charcoal on drawing paper, especially visible in hair and fabric regions.

7.5 Pop Art Filter

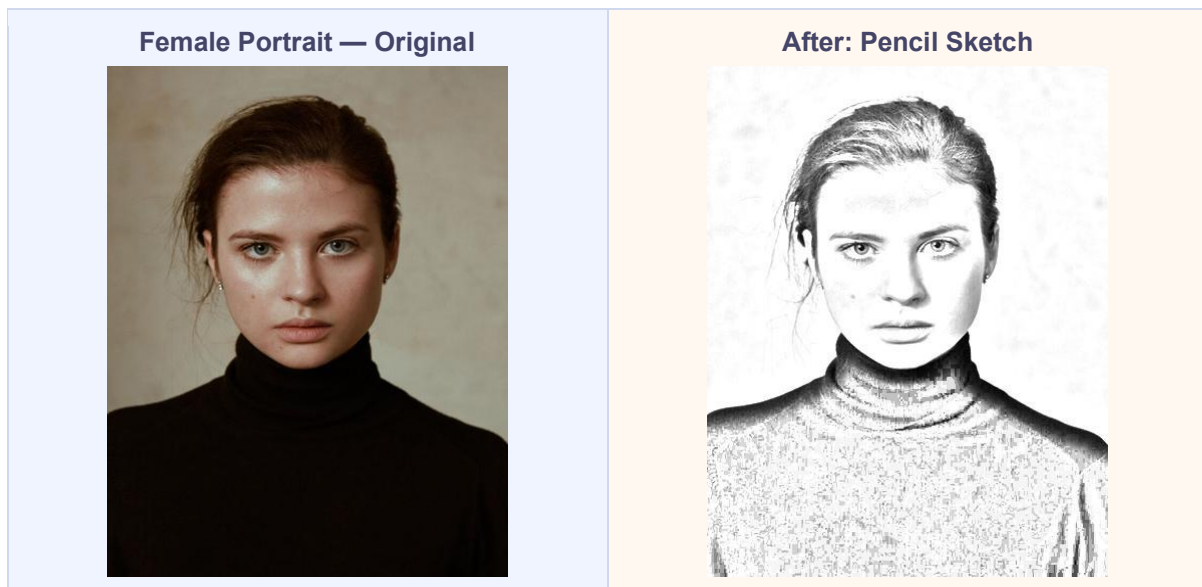
PORTRAIT SAMPLE — POP ART EFFECT



The 5-level quantization creates the distinctive flat color zones of pop art while the black Canny outlines reinforce the comic-book aesthetic. The final contrast stretch ensures vivid, saturated colors that are visually striking and instantly recognizable as the pop art style.

7.6 Multi-Sample Comparison

DIFFERENT SUBJECTS — PENCIL SKETCH VS OIL PAINTING



Male Portrait — Original



After: Oil Painting



The filters perform consistently across different subjects, lighting conditions, and facial structures. The pencil sketch preserves fine texture details while oil painting creates smooth, painterly regions in both female and male portrait subjects.

8. Code Structure

The entire application is implemented as a single MATLAB classdef file: DIP.mlapp (compiled from DIP.m + App Designer layout). The class inherits from matlab.apps.AppBase and is organized as follows:

Section	Description
<code>properties</code> <code>(Access=public)</code>	Declares all 6 UI component handles: UIFigure, DropDown, DropDownLabel, UploadImageButton, UIAxes, UIAxes2
<code>DropDownValueChanged()</code>	Core processing callback: reads stored image → runs selected filter algorithm → displays result in UIAxes2
<code>UploadImageButtonPushed()</code>	File picker → imread → stores in UserData → displays in UIAxes
<code>UIFigureButtonDown()</code>	Empty placeholder for potential future click-to-process interaction
<code>createComponents()</code>	Initializes all UI elements with positions, colors, event bindings, and dropdown items
<code>DIP()</code> constructor	Entry point: calls createComponents → registerApp → returns app handle
<code>delete()</code>	Cleanup method: deletes UIFigure on app shutdown

8.1 Key Design Decisions

- **UserData Storage:** The uploaded image is stored in UIFigure.UserData rather than a class property, avoiding state management complexity. The filter callback always reads from this central store.
- **im2double Conversion:** All filter algorithms work on [0,1] double-precision arrays for mathematical stability. Results are implicitly converted back for imshow display.
- **Switch-Case Routing:** Filter selection uses a clean switch-case structure, making it easy to add new artistic styles in future without changing the overall control flow.
- **Guard Clause:** DropDownValueChanged returns immediately if UserData is empty (no image uploaded), preventing null reference errors.

9. Conclusion

Artistic Image Studio successfully demonstrates the practical application of Digital Image Processing concepts in a real-world interactive application. By combining classical DIP techniques — including Gaussian filtering, Canny edge detection, morphological operations, bilateral filtering, and intensity transformations — the application transforms ordinary photographs into visually compelling artistic renditions.

The project achieves all its stated objectives: five distinct artistic filters are implemented and functional, the MATLAB App Designer GUI provides an intuitive dual-panel interface for real-time comparison, and the underlying algorithms correctly apply DIP principles studied in the course.

The before-and-after results demonstrate that the filters produce consistently high-quality outputs across different input images, including varied lighting conditions, skin tones, and subject types. Each filter has been fine-tuned with appropriate parameters to maximize artistic resemblance to its namesake medium.

Future Enhancements

- Add image export functionality to save the processed output to disk
- Implement adjustable filter parameters via sliders (e.g., sketch detail level, oil paint smoothness)
- Add additional artistic styles such as Watercolor, Mosaic, HDR Tonemapping, or Vintage Film effects
- Introduce batch processing capability to apply filters to multiple images at once
- Add a histogram display panel to visualize intensity distribution changes before and after processing
- Implement undo/redo functionality for iterative artistic exploration

References

- Gonzalez, R. C., & Woods, R. E. — Digital Image Processing, 4th Edition, Pearson (2018)
- MATLAB Image Processing Toolbox Documentation — MathWorks (R2021a)
- MATLAB App Designer Documentation — MathWorks: Getting Started with App Designer
- Canny, J. — A Computational Approach to Edge Detection, IEEE TPAMI, 8(6), 679-698 (1986)
- Tomasi, C. & Manduchi, R. — Bilateral Filtering for Gray and Color Images, ICCV (1998)

